COS 214 Practical 6

TIVIDY: Workflow Management System

Team details(The Seven Bits):
Chavonne Makurira u25020880
Felicity Moyo: u24702791
Nico Sibiya u24667642
Nicole Bare u25443705
Heinrich Klopper u25030932
Tafadzwa u25370325 Manyengawana
Deon-Moller Steenkamp u25135742

Github repository link: https://github.com/ChavonneCodes/COS214-Prac6

TASK 1: Research Workflow Management Systems

A workflow management system (WFMS) is a software system that manages the flow of work between users according to formal specifications of business processes called workflows. A workflow specifies what tasks need to be performed and decides in what order they need to be executed in. Furthermore, it decides how data flows internally between the tasks and the applications needed for the tasks to be carried out. An example is a travel expense reimbursement workflow will specify the tasks of filling,checking and signing the form and reimbursing the amount of money needed to be reimbursed. The workflow specifies that before the amount is reimbursed the form first needs to be signed. It also defines the flow of the expense form between the tasks and the spreadsheet application. In addition to that for each task of the workflow, a rule needs to be set that specifies the responsible and capable users are able to carry out the task.

A workflow definition is the static specification of the process or the template of a process, activity or an automatic part of a manual activity. It is built from available building blocks and once the processes and activities have been defined, the workflow system can assist in the execution of these processes. However, a running workflow instance is for a particular execution of a process. A workflow definition would be an object class and the workflow instance would be the instance of that class.

Activities or work items are modelled and represented at two different levels. At definition level an activity describes a logical step in the process-its name, required role, preconditions and routing. It is a reusable description shared by many instances. At runtime, each activity in a running instance is represented by a work item , which combines the activity with a specific case. The work item has its own state, assigned participant, data and history.

Work items could be assigned and routed by evenly distributing work among all available resources or give the waiting work to a resource with the shortest queue or manually implement assignments, however this is not optimized. The WFMS needs to know what is available to perform the work and have a profile about each user which includes what each resource is qualified to do and how good it is at that type of work and whether the supervisor wants the work to be assigned to them. As for routing there are two types of routing which is control flow routing and resource routing, Control flow routing defines the path through the process which could be sequential,parallel or conditional. Resource routing deals with which participant receives the work item which could be role based, workload based and rule based. More advanced systems use dynamic routing which results in escalation when deadlines are missed and delegation to another qualified member. A well designed workflow engine separates these two so they can be changed independently.
In a workflow management system approvals can be handled by routing a work item to one or more authorised participants for a formal decision. It could be one approval participant or sent through many participants for approval or multiple participants could decide simultaneously. Rejections could terminate or loop back for reprocessing or trigger a response. Lastly in the case of Escalation it could be triggered by a time limit which uses hierarchical routing to route the work item to a supervisor participant which may have a maximum number of escalation levels.

Roles are represented by assigning a role or roles needed for a step. Roles are abstract job functions. Participants are the real people or systems assigned to those roles at runtime and the responsibilities are the tasks the role/s or participant/s are expected to do.

Dependencies are handled by defining the order in which work items must happen which are implemented by conditional logic. Events are handled by having event listeners and real time observability. Failures are handled by rollbacks, retrying or aborting the process.

Workflow management systems usually communicate to external systems using APIs and adapters because it needs to get something it doesn't have on its own.
Workflow history and audit information are recorded by capturing meaningful action,state change and decisions that happen during a workflow instance and storing them in a timestamped log which is usually stored in an append-only structure.

Running workflows are managed by watching several live things such as the status of each instance, progress, bottlenecks, errors and failures and time metrics. Reporting usually happens with dashboards that summarize what has happened over time and notifying observing participants of any changes.

A workflow engine is reusable across different processes if it only contains generic logic and if it pushes all process specific details into data or configuration. The engine should know how to run a workflow but not know what the workflow is. It should have configurable variation points.The workflow definition should have information such as the process structure, order and routing, roles, rules, data schema and transitions. The workflow instance contains the actual data, state and history for one specific case.

Real workflow systems model nested activities and sub-processes by embedding sub processes in a tree structure or by call activity where the parent process calls a separate

process.Configurable decisions in a workflow system are assignment, routing, approval, escalation, deadlines, notifications, conditions and integration.Changes to workflow definition while work is already in process is handled by versioning with instance pinning. The running instance never changes its definition. It keeps using the version it started with and a new version is created for the new definition.

References:

[1]C.J.Bussler,"Policy Resolution for Workflow Systems", Digital Technical Journall,vol.6,no.4,pp.26-49,1994.[Online]. Available: https://shiftleft.com/mirrors/www.hpl.hp.com/hpjournal/dtj/vol6num4/vol6num4art2.pdf

[2]C.A.Ellis,"Workflow technology,",in Computer Supported Co-operative Work,M. Beaudouin-Lafon, Ed.Chichester,U.K.:John Wiley & Sons,1999, pp.29-52.[Online]. Available: https://d1wqtxts1xzle7.cloudfront.net/30913450/computer_supported_co-operative_work-libre.pdf?1392121767=&response-content-disposition=inline%3B+filename%3DSoftware_infrastructures.pdf&Expires=1791076615&Signature=UxHx1GvySkXUEpp8LglD0xI3MbDIoN2Afpjj~an7YAB18tSWUczLYrTr6xM53AmYeeBt4HaZ8yti-k4DhziUGlTV7szKEZtmzfnHUtPTRavOeMvvEehTDG~DSKZ9OmCh6AcwcS3JJiiW32PcHkMnRAZBUlxPgC-iX5qZoVpQpOcDAXLwDFEQk4FkohpS3EN8ekQGx5N0guyIkfJZuVmoXoJbMtIyfKwjiIJY68vIIypfuRl6kZj9g~3~xT~MptHcYibO68sQo9LkzmCH~YPppK9G8KO2gYoildvGNJ2OGj7vKMEY6IUvp5C-qh1ECRxAZyqkb5Ap5h8a-XBiYBMgeQ__&Key-Pair-Id=APKAJLOHF5GGSLRBV4ZA#page=43

[3]C.Plesums,"Introduction to Workflow," in Workflow Handbook 2002,L.Fischer, Ed.Lighthouse Point, FL, USA: Future Strategies INC., 20022, PP.16-38. [Online].Available: https://pages.di.unipi.it/ferrari/CORSI/SISD/WorkFlowIntro.pdf

[4] Microsoft, "Configure approval processes in a workflow," Microsoft Learn. Accessed: Oct. 4, 2026. [Online]. Available: https://learn.microsoft.com/en-ca/dynamics365/fin-ops-core/fin-ops/organization-administration/configure-approval-process-workflow

TASK 2: Define the Scenario and System

TIVIDY is a generic workflow management system which controls the flow of processes from start to finish. Clients specify workflows suited to their needs,manage them and execute them as instances using the system. Due to the generic nature of TIVIDY, clients are not limited to a single workflow but may define multiple workflows which differ in structure, approaches, routing and overall execution. The workflow management system is not responsible for carrying out the actual work (logic) behind the workflow; it is only used to define the sequence of activities that must take place, how they must be completed and who is responsible for what.

Using the workflow manager, clients define the scope of their workflows(workflow definition), which includes activities and their ordering, routing and decision conditions, participants and roles, rules regarding approval and rejections, state transitions, dependencies and constraints.The workflow definition is generic and contains no specific data such as participant names. Workflow instances are created by TIVIDY using the workflow definition. Instances contain meaningful data such as the current state of the workflow, assigned participants, specific work items, decisions that have been made, history of the instance, as well as information specific to the instance. Multiple instances may be created from the same definition. Within the system instances are kept independent and thus changes to one instance will not affect another.

Workflow scenario 1: Project approval

Process:

The chosen scenario used to demonstrate the application of TIVIDY is a project approval workflow in a technology company. The technology company is responsible for creating software solutions as products and services. Before the start of a project the company receives detailed proposals from product teams who communicate with clients. It is important that these proposals are technically and financially feasible thus they must undergo a series of approvals,reviews and changes before execution. The technology company is large-scale and thus multiple proposals are handled simultaneously.

The process being managed is the lifecycle of the project proposal from the time it is submitted to the time it is approved/ rejected. The workflow includes four major participants. The product manager - reviews the proposal submitted by product teams for correctness and completeness. The head software engineer - determines if the project is technically feasible. The head of finance - decides whether the project is profitable and financially feasible. The chief operation officer-provides the final decision of whether the proposal is approved or rejected. Lower level employees may perform work items such as comparing profitability of previous projects to assist major participants in decision making.

Branching:

Work items may include, submitting a proposal, adding priority to a proposal, review of correctness and completeness, budget comparison ,financial feasibility review, risk and security evaluation, technical review,creation of summary report, final review, and beginning project.

Routing is a major part of this workflow. In terms of control flow routing sequential and conditional routing is utilized. The proposal must be reviewed by at most one participant at a time and may only be passed on to the next participant if approved, otherwise rejection/ change requests would redirect the proposal to previously visited participants. On the other hand, in terms of resource routing, role-based routing has been chosen . Work items are assigned to participants based on their job title.

Workflow branches off of the input data being the budget amount set from the “Budget Audit” definition. After review if the budget is over limit it routes to Executive Approval and if it is within_limit then it skips the executive step and routes directly to Project Kickoff.

Participants:

Participants are responsible for approvals. Whether or not the proposal is passed on to the next participant is dependent on the decision of the participant currently holding the

proposal. For example the technical review is dependent on the approval of the completeness review. The financial review is dependent on the technical review approval etc. We have Deon the proposer,Lerato the Product Manager, Thandi the Head Engineer, Zanele the Analyst,Ayesha the Finance Officer and Naledi the Executive.

Failure Paths(Exceptions):
The proposal can undergo multiple failures/ exceptions during the workflow instance. The proposal may be incomplete and rejected during the completeness review by the product manager. It may be decided that the project exceeds the budget and is rejected by the head of finance. Due to prioritization of other proposals the approval deadline of the proposal may be missed. The Executive Approval step may miss its deadline and the Chain of Responsibility will handle the timeout and automatically escalate the approval to the CEO reassigning the work item.

Outcomes:
The workflow may have two outcomes. In the best case the proposal is reviewed,audited and approved by all departments and reaches the state “approved” and is therefore given the greenlight to proceed with implementation. In the second outcome the proposal may be rejected by one of the departments, even after reviews and changes it may be deemed as “rejected” and will not be revisited. This prevents the project from starting. Both outcomes result in an audit trail.

Workflow scenario 2: Project approval

Organization or environment: University’s internal IT Service desk for maintaining digital infrastructure for students or staff.

The process being managed: The incident management workflow for a campus learning portal outage. The process begins when an outage is reported,moves through diagnosis and repair, and requires a verification step before the incident is closed.

Participants: Student(Guest) who reports the issue but has no authority to modify the workflow, a technician who diagnoses the outage and tests student access after repairs and a service administrator that performs the actual configuration repairs and formally closes the incident

Major pieces of work: Diagnose the outage, restore the portal configuration and test student access and close the incident.

Important decisions and routing points:
Conditional routing:
After the “Test student access” task, workflow routes based on outcome. If passed,it routes to Close. If failed,it loops back to “Restore the portal configuration”

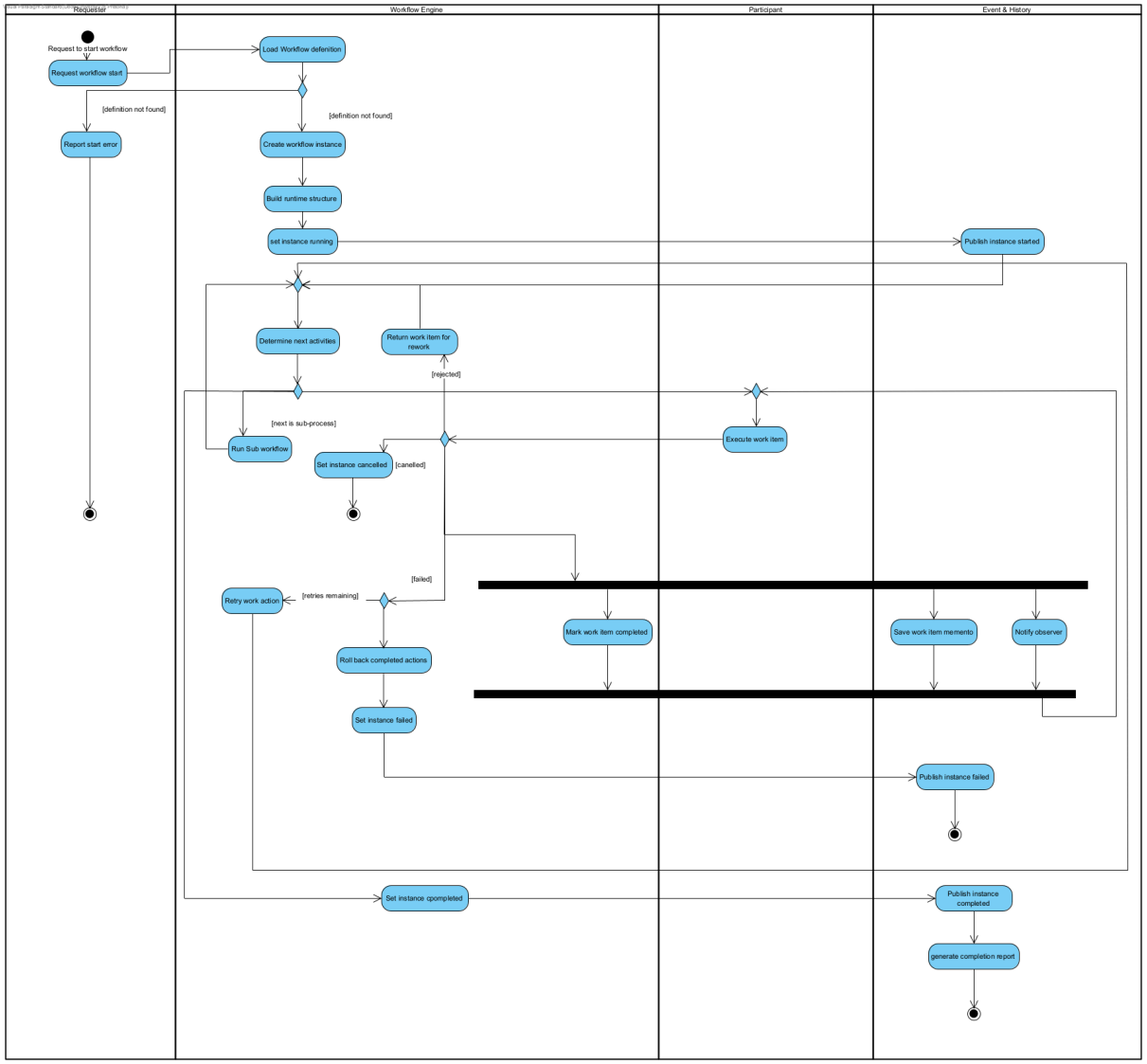
Resource routing: Workload based assignment ensures the least busy technician or administrator picks up the task.

Approvals and dependencies:
Chain of Responsibility:
Dependencies: Repair cannot happen until diagnosis is complete,verification cannot happen until the repair is complete.

Possible failures or exceptions:
Unauthorized action: A student attempts to update the workflow state which is blocked by the security decorator
Invalid state: An administrator requests an illegal state

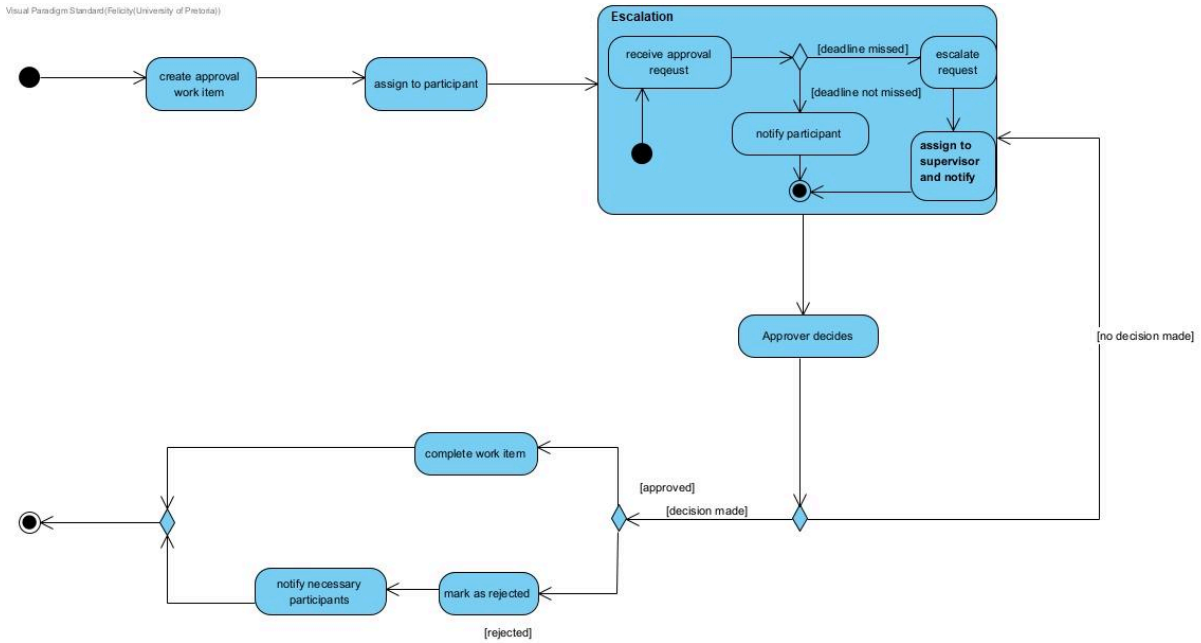
TASK 3: UML Activity diagrams

1.



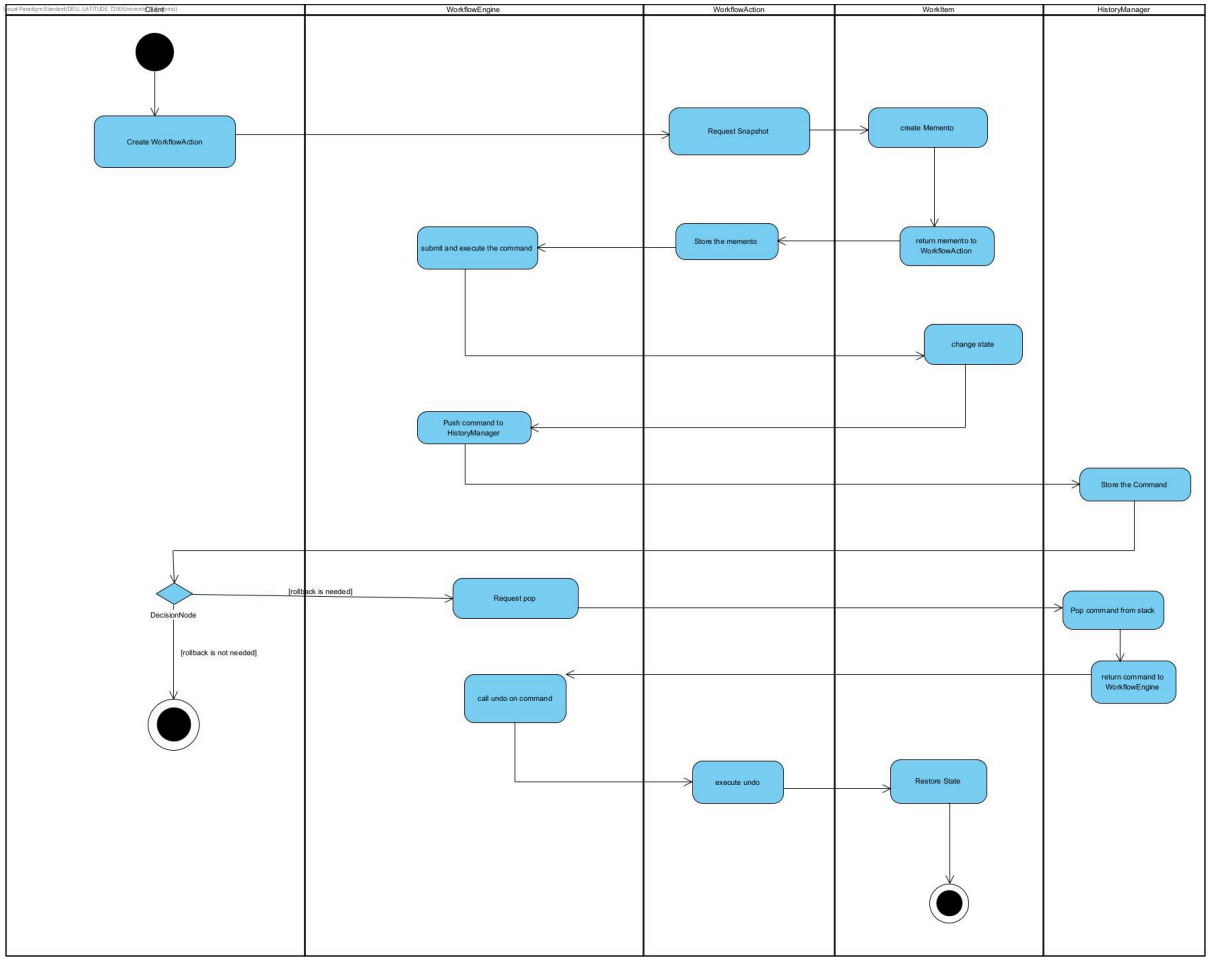
link:https://github.com/ChavonneCodes/COS214-Prac6/blob/main/docs/Activity%20Diagram1.png

2.



link:https://github.com/ChavonneCodes/COS214-Prac6/blob/main/docs/Activity%20Diagram%202%20prac6.jpg

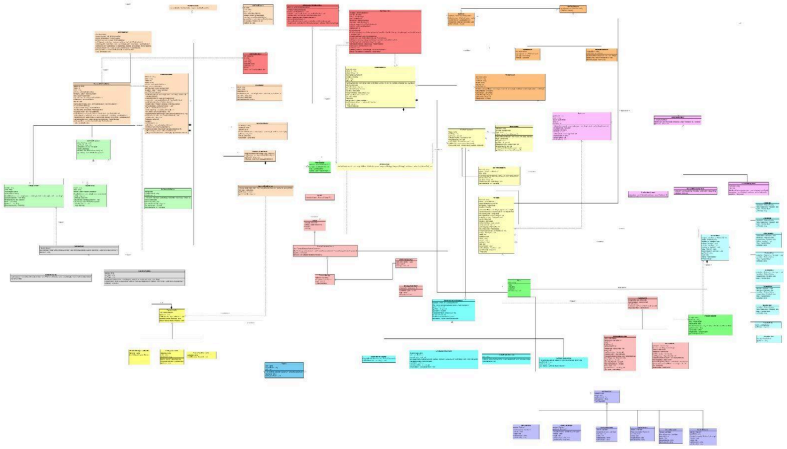
3.



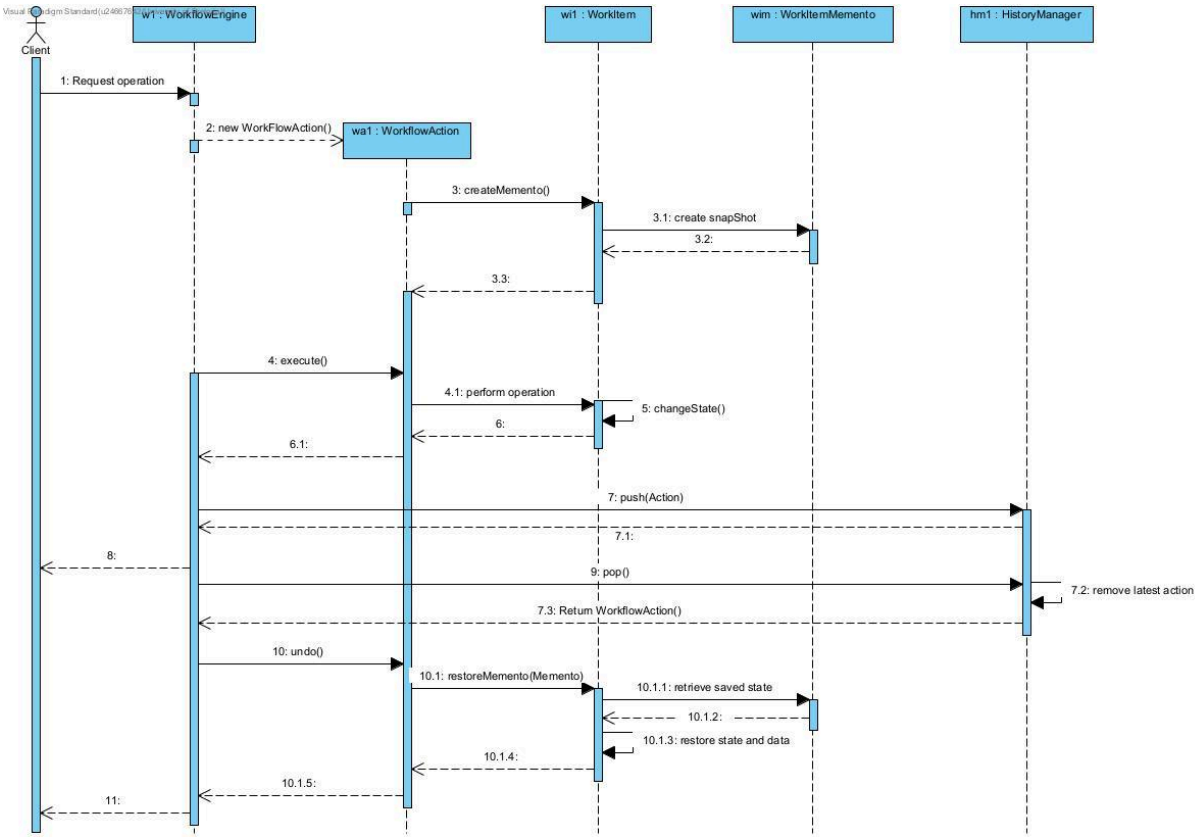
link:https://github.com/ChavonneCodes/COS214-Prac6/blob/main/docs/Activity%203-Memento%20and%20Rollback.jpg

TASK 4: UML Class Diagram

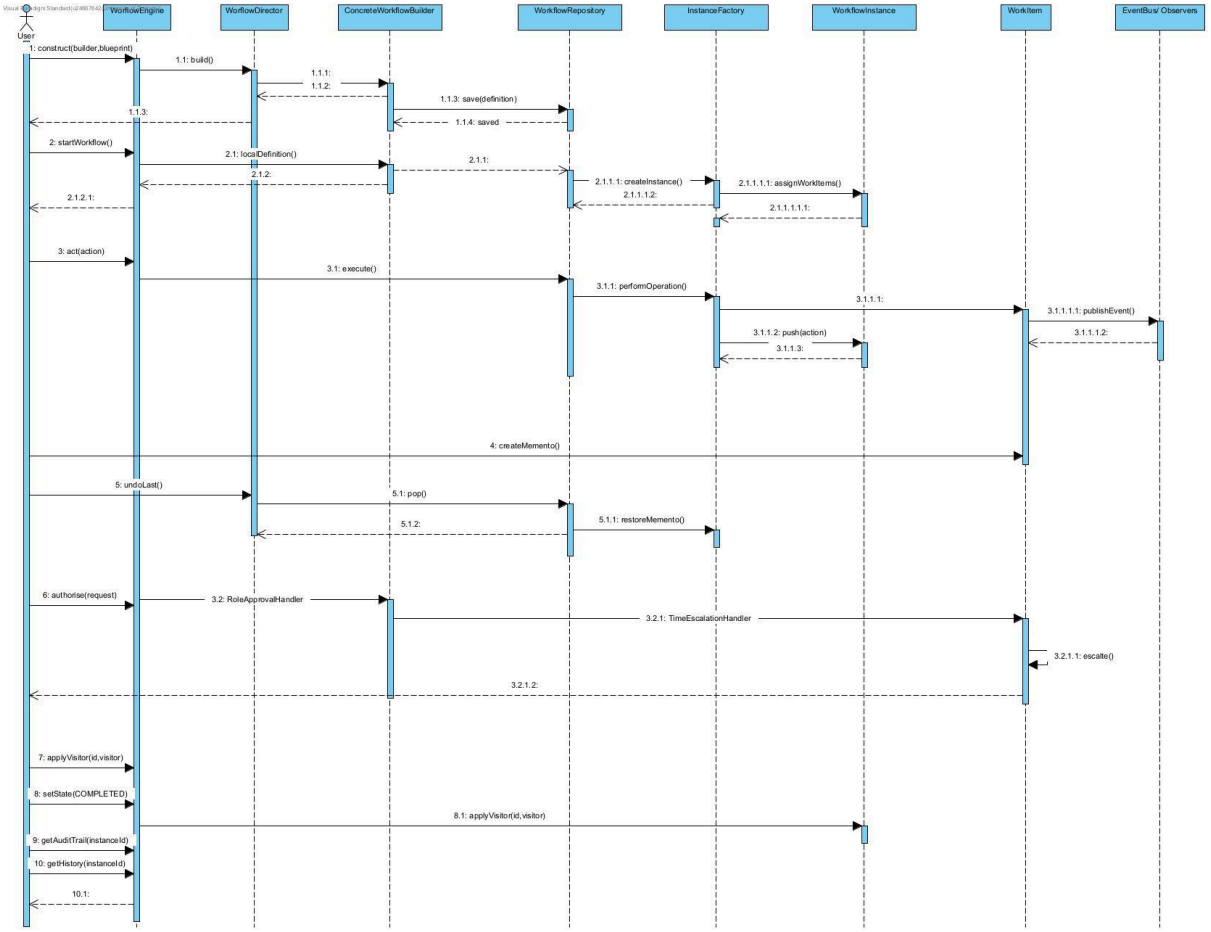
link:https://github.com/ChavonneCodes/COS214-Prac6/blob/main/docs/UmlV7-Final.jpg



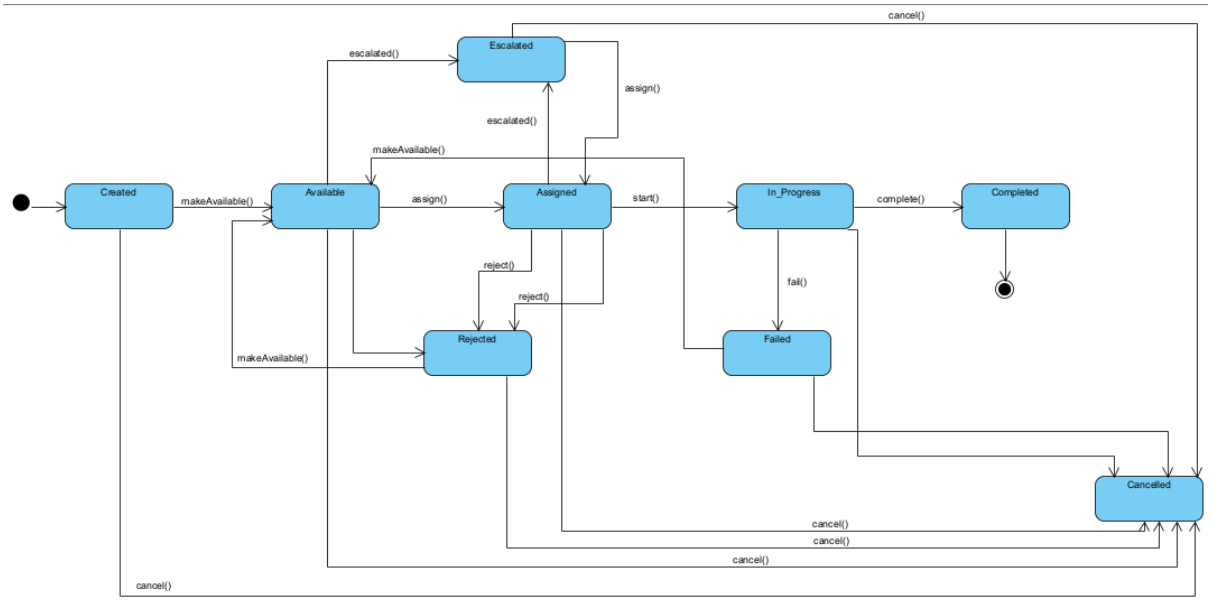
TASK 6:runtime behaviour diagrams



Link: https://github.com/ChavonneCodes/COS214-Prac6/blob/main/docs/Memento%20Restoration.jpg



link:https://github.com/ChavonneCodes/COS214-Prac6/blob/main/docs/Pipeline%20Execution.jpg



link:https://github.com/ChavonneCodes/COS214-Prac6/blob/main/docs/Prac6_State.png

TASK 7:Design decisions and revisions

UML Improvements:

No overlapping lines, make it neater

Remove the blocks and add notes

Put functions in assignment strategy

Consistent arrows

State instead of enums

Bridge separates the implementation from the representation, different types of workflows

Put void when it does not return anything

Sequence diagram improvements:

Return arrows must not have anything written

Use the functions and not notes like words.

Activity diagrams improvements:

leave the diamonds empty and put the full guard in the square brackets and don't have overlapping lines.

State diagram improvements:

Use visual paradigm and move the labels and no overlapping lines

To do for the next meeting: Implement concrete skeleton and basic interactions. Generalized implementation.

Scope

Tividy is a generic workflow management system with the following scope:

1) Responsibilities: to allow users of the framework to define and run a set of steps representing a particular workflow; to track the state of each step; to provide a framework for approval of steps; to handle events triggered during the processing of a workflow; and to maintain an audit trail of each workflow.
2) Boundaries: the system only tracks the work to be done, rather than doing this work itself; the system does not persist any data; the system does not perform authentication; and the system does not contain scenario-specific business rules.

Design rationale

TIVIDY intends to be a robust and generic framework that allows organisations to define process flows unique to their particular business problems. It is deliberately agnostic of any particular scenario and extensible in order to accommodate a wide variety of business processes.

Important Design decisions

1. Separate workflows definitions from running workflow instances
   Many problems have to run at the same time, a change to one running case must not affect others
   - A workflow definition describes how a process must behave, a workflow instance represents one specific execution with its own data state and participants, Many instances can be created from one definition
2. Pin each running instance to a definition version
   A definition can change while a process is running
   - WorkflowDefinition stores the version, workflowInstance stores a definitionVersion, running instances keeps the version and new instances under the new ones
3. Representing nested work as tree structures
   Processes and instances are stored in sub trees so that the engine must walk through the trees without knowing how children are stored

   - Definition side: WorkflowComponent, with StageDefinition as container and ActivityDefinition / SubProcessDefinition as leaves.
   - Runtime side: RuntimeComponent, with StageInstance / SubProcessInstance as containers and WorkItem as leaf. RunTimeIterator / DepthFirstIterator walk the runtime tree.
4. Separating what comes next from who does it with the strategy pattern
   What comes next in the workflow and who does it does not mean that the process must be re-written. Each Activity definition stores two strategies that store what comes next and decides who does it in the other strategy
5. Model lifecycles as state objects using the state pattern
   Allowed actions depend on the current state.
   - WorkItem uses the concrete patterns to decide what actions are possible based on the current state
6. Turning actions into objects and keeping the history
   Supporting what happened and undo functions using the memento pattern
   - Each action has a WorkflowAction that has undo and start functions to guarantee rollback possibility

7. Keep reaction events out of the engine
   Auditing and notifications act upon the same events
   - Listeners will be used to separate this without changing the engine

GOF Participant Mapping

Pattern	GoF participants in TIVIDY	Problem solved / justification
State	**Context:** WorkItem **State:** WorkItemState **ConcreteStates:** AvailableState, AssignedState, InProgressState, CompletedState, FailedState, RejectedState, EscalatedState	A work item behaves differently depending on its lifecycle stage. Operations that make sense while a task is available or assigned may be invalid after it has completed, failed or been rejected. Rather than placing a large switch on the current status inside WorkItem, behaviour is delegated to a WorkItemState. The current state can therefore change at runtime while WorkItem retains the same interface. This directly supports the lifecycle behaviour required of running workflow items.
Observer	**Subject:** EventBus **Observer:** WorkflowObserver **ConcreteObservers:** AuditObserver, NotificationObserver **Event:** WorkflowEvent	Multiple parts of TIVIDY may need to react when a workflow event occurs. For example, completion of a work item may need to be recorded in an audit history and may also trigger a notification. EventBus can notify registered WorkflowObservers without the workflow engine knowing the concrete systems receiving the event. The set of event consumers can therefore vary independently of workflow execution.
Iterator	**Iterator:** RunTimeIterator **ConcreteIterator:** DepthFirstIterator **Aggregate:** runtime workflow/component structure	Running workflows contain nested stages, subprocesses and work items. Clients such as monitoring, reporting or the workflow engine should be able to traverse this structure without depending on how its children are stored internally. RunTimeIterator defines the traversal interface while DepthFirstIterator supplies a particular traversal algorithm. Additional traversal strategies can later be introduced without changing the workflow components themselves.
Builder	**Builder:** WorkflowBuilder **ConcreteBuilder:** DesignWorkflowBuilder **Director:** WorkflowDirector **Product:** WorkflowDefinition	A workflow definition may contain multiple stages, activities, routing rules, subprocesses and other configuration. Constructing the entire definition directly would tightly couple clients to its representation and require a large constructor. Builder separates the construction process from the completed WorkflowDefinition. WorkflowDirector controls construction while a concrete builder creates the actual workflow representation. Different construction procedures can therefore be supported without redesigning WorkflowDefinition.
Composite	**Component:** RuntimeComponent **Composites:** StageInstance, SubProcessInstance **Leaf:** WorkItem	Workflow work is inherently hierarchical. A stage can contain activities or subprocesses, and a subprocess may itself contain further work. Composite allows individual WorkItems and groups of runtime components to be handled through a common RuntimeComponent interface. The workflow engine can therefore operate on nested structures without repeatedly checking whether an object is an individual task or a container.
Strategy – Assignment	**Strategy:** AssignmentStrategy **ConcreteStrategies:** RoleBasedAssignment, WorkloadBasedAssignment, RuleBasedAssignment **Context:** workflow/activity assignment logic	Different organisations may decide who receives work using different rules. Some workflows may assign according to role, others according to current workload, while others may evaluate workflow-specific rules. These algorithms should not be hard-coded into WorkItem or the workflow engine. AssignmentStrategy makes the algorithm replaceable while preserving the rest of the execution model.
Strategy – Routing	**Strategy:** RoutingStrategy **ConcreteStrategies:** SequentialRouting, ConditionalRouting **Context:** activity/stage progression logic	Not all workflows determine the next activity in the same way. Some follow a fixed sequence while others choose a route according to conditions or workflow data. Encapsulating routing in RoutingStrategy allows a workflow definition to select an appropriate routing rule without changing the workflow engine. The routing algorithm is therefore the part expected to vary.
Chain of Responsibility	**Handler:** Team **ConcreteHandlers:** StaffTeam, MidLevelManagementTeam,	Approval and escalation may require a request to move through different levels of responsibility until a participant is authorised to handle it. Each team can either process a request or forward it to the next handler. This

Pattern	GoF participants in TIVIDY	Problem solved / justification
	ExecutiveTeam **Request:** Request / approval request	avoids coupling the requester to a specific approval level and allows the approval hierarchy to be changed independently of the code issuing the request.
Memento	**Originator:** WorkItem **Memento:** WorkflowMemento **Caretaker:** HistoryManager	TIVIDY needs to retain meaningful workflow history and may need to restore earlier runtime state. WorkItem can capture its relevant state inside a WorkflowMemento, while HistoryManager stores mementos without needing to understand their internal state. This preserves encapsulation while providing a basis for rollback, restoration and historical state tracking.
Command	**Command:** WorkflowCommand **ConcreteCommands:** CreateDesignCommand, ValidateDesignCommand, StartCommand, CompleteCommand, CancelCommand, RejectCommand, AssignCommand and other workflow operations shown in the design **Receiver:** workflow engine/runtime objects	Workflow actions such as starting, assigning, completing or cancelling work are represented as objects rather than direct calls. This allows requests to be executed consistently and gives TIVIDY a place to store descriptions and, where appropriate, undo behaviour. New operations can be introduced by adding command classes rather than expanding a central switch statement in the workflow engine.
Decorator	**Component:** RuntimeComponent **Decorator:** RuntimeComponentDecorator **ConcreteDecorators:** PriorityDecorator,SecurityDecorator, LabelDecorator,LoggingRuntimeDecorator	Some workflow operations may require additional behaviour such as having priority or security handling without changing the original operation. This allows decorators to be added to the composite workflow definition tree to add additional responsibilities without having to add additional functions or processes to determine whether it is a decorator or leaf or composite.
Visitor	**Visitor:** RuntimeVisitor **ConcreteVisitor:**ProgressReportVisitor,WorkloadVisitor **Element:** RuntimeComponent **ConcreteElement:**WorkItem,StageInstance, SubProcessInstance	We have a composite tree that has 3 node types that needs to be iterated over such as progress reports and workload summaries but we do not want those operations living in those node classes. The node classes do not have to be changed and each visitor is an operation to be performed on the ConcreteElements.
Facade	**Facade:** WorkflowEngine	Allows a user of the system to use a simplified interface without having to do directly use or under the subsystem while still providing access to the client to use the subsystems

Design decision history

The initial design included the adapter pattern,however we realised that it was unnecessary because the adapter would only be used for external systems which would be too specific for a generic system or it would be used to change the system slightly but since the design has just been created there is nothing to adapt.

We initially did not have a Visitor pattern but we included it so that the whole instantiated tree could be traversed so the client can collect and see data based on it and perform multiple calculations on each of the elements being the StageInstance,SubProcessInstance and WorkItem.

We added the prototype pattern because we realised we would need to implement cloning with versioning so different definitions can be created with minor changes without having to change the original version.

Implementation explanation

The TIVIDY engine is implemented in C+11 and built with a plain Makefile. The build produces two primary executables-main(the two-scenario demo) and run_tests which is the GoogleTest suite. The same file is used in the Docker image ensuring that the code compiled during CI matches what developers run on their machines. The image is built from ubuntu:22.04 and installs GoogleTest 1.12.1, the last release that supports C+11 language level required by the brief.

Build//Run/Test instructions

Build:

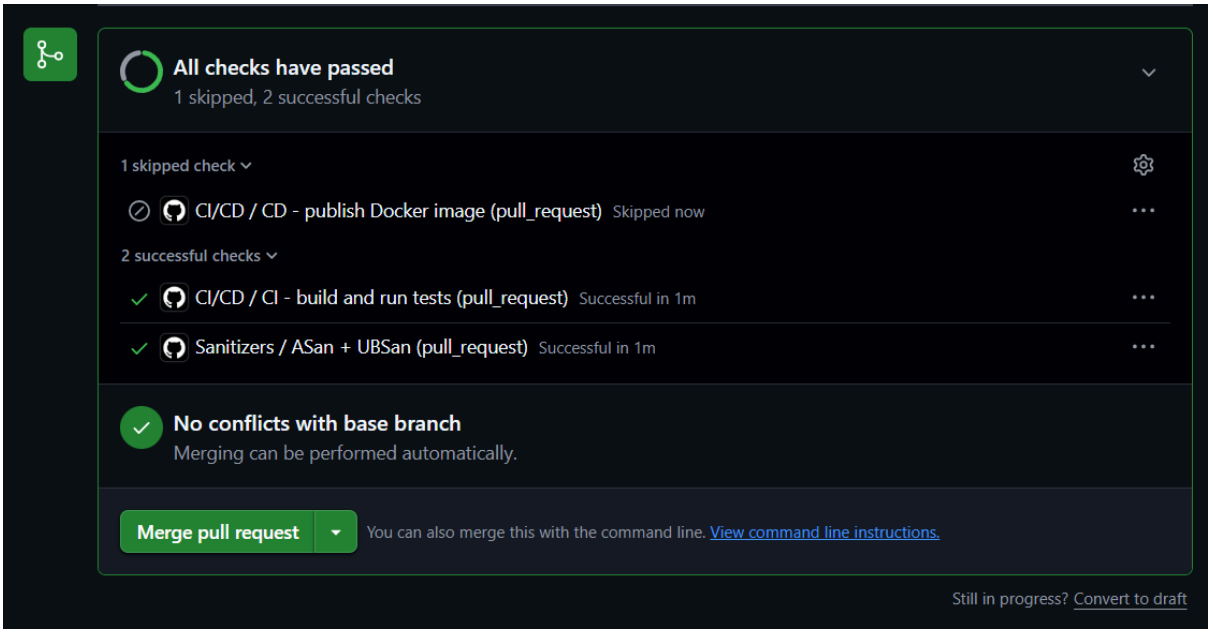
From the root repository run:
Make clean
Make all
OR
use docker by running docker build -t tividy .

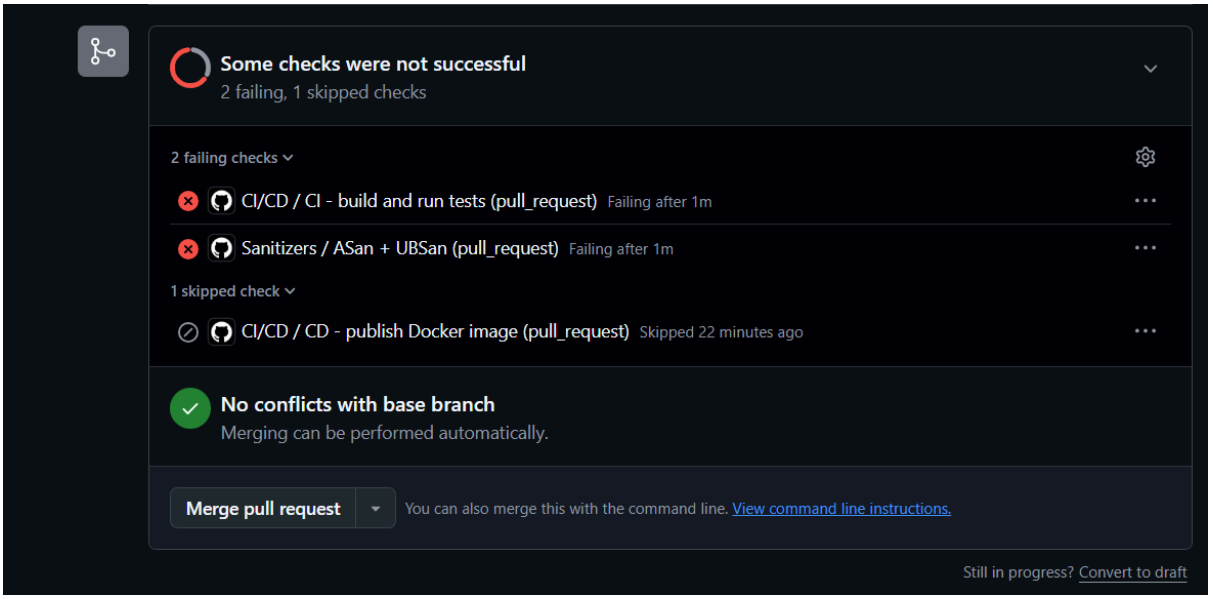
Run:

Run the tests: docker run - -rm tividy ./run_tests OR make tests
Run two scenario demo: make live OR docker - - rm tividy ./main
Doxygen: doxygen Doxyfile
/run_tests

Engineering evidence

CI/CD pipeline and automated quality checks

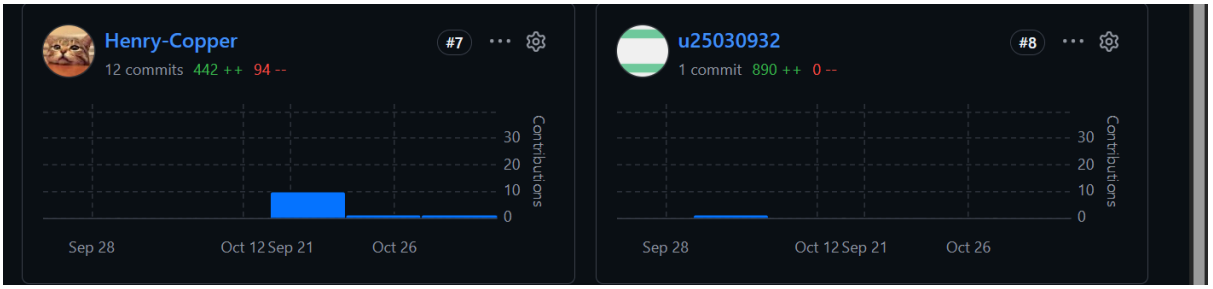




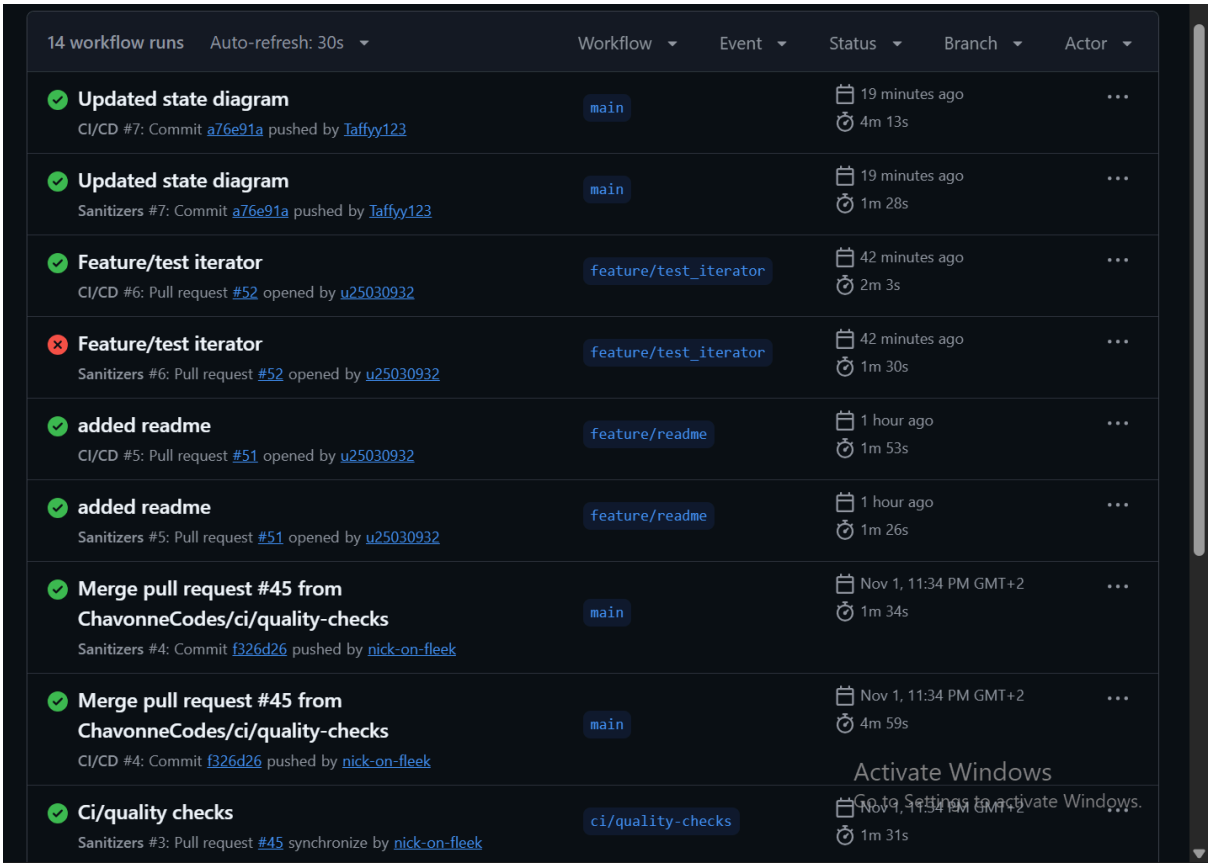
Git proof

Commits and contributions:

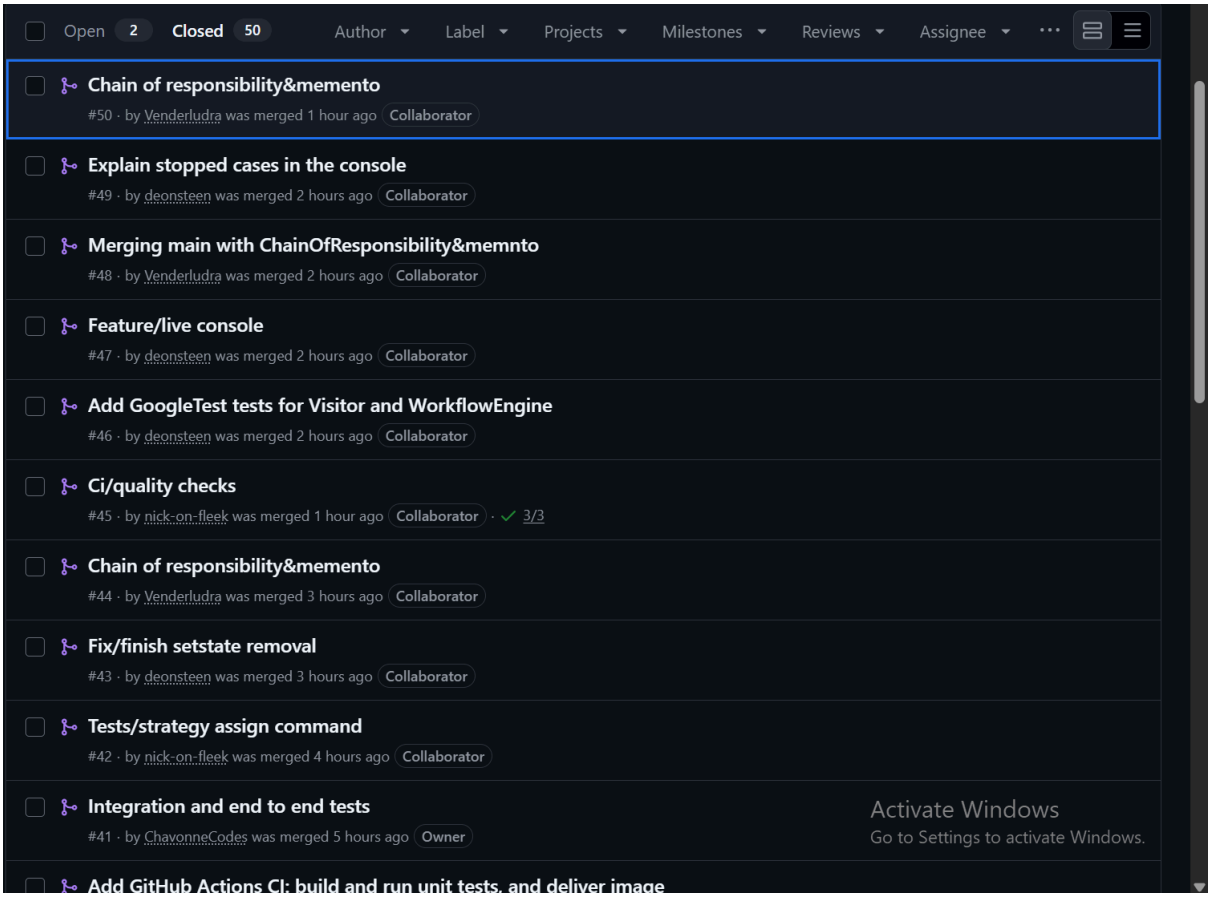




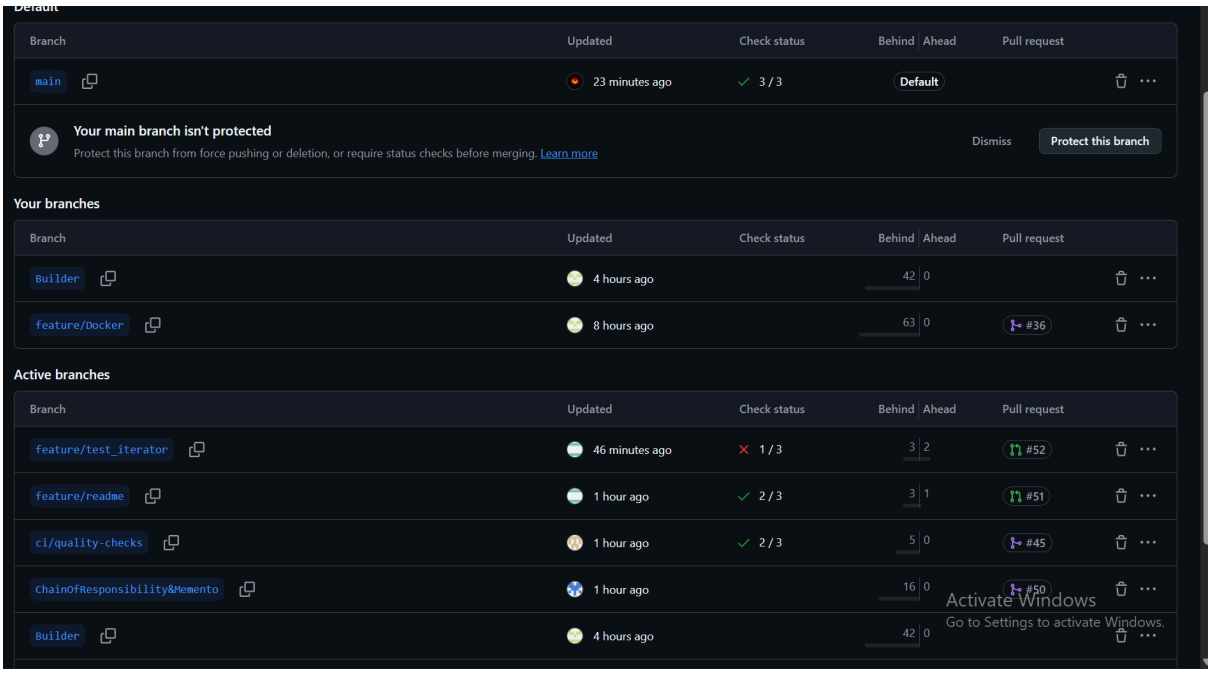
CI/CD pipeline:



Pull requests:



Multiple branches:



How we used github

Every feature and pattern that we implemented was done on a separate branch and tested on that branch using the most recent version of code from main by pulling from main. When it was completed a pull request to merge with main was sent in for review by other collaborators and once approved it was merged into main.

Team Contribution Statement

Member 1: Nico Sibiya

- Contributions: Two sequence diagrams,Memento,Chain of Responsibility
- Communication participation: Attended all meetings, responsive, maintained great communication

Member 2: Nicole Bare

- Contributions: UML, Prototype,Strategy,Observer,CI pipeline,automated quality testing
- Communication participation: Attended all meetings, responsive, maintained great communication

Member 3: Felicity Moyo

- Contributions: task 2, one activity diagram, contribution statement, State,Command, Doxygen file,Powerpoint
- Communication participation: Attended all meetings, responsive, maintained great communication

Member 4: Chavonne Makurira

- Contributions: task 1, one activity diagram, UML,Builder,Composite,Integration testing, Report compilation
- Communication participation: Attended all meetings, responsive, maintained great communication

Member 5: Deon-Möller Steenkamp

- Contributions: one activity diagram, GUI, Facade,demo main
- Communication participation: Attended all meetings, responsive, maintained great communication

Member 6: Heinrich Klopper

- Contributions: UML, system scope and design rationale, Iterator,README,Dockerfile
- Communication participation: Attended all meetings, responsive, maintained great communication

Member 7: Tafadzwa Manyengawana

- Contributions: state diagram, GOF participants and pattern justifications,Decorator,demo main
- Communication participation: Attended all meetings, responsive, maintained great communication